Branch Decision Guide — Do We Need This?

Generated: 2026-03-20 | For: James Spalding
Each branch was inspired by an open-source project. Here's what that project ACTUALLY does, what we TOOK from it, whether we NEED it, and the honest verdict.

---

1. `feature/cashclaw-knowledge-system`

Original Project: CashClaw — An autonomous agent that finds freelance gigs, does the work, gets paid, and improves from feedback. It's a money-making machine.

What CashClaw actually does: Monitors freelance marketplaces, auto-quotes jobs, executes tasks via LLM, delivers results, collects crypto payment, uses client ratings to improve. It literally earns money autonomously.

What we took: NOT the money-making part. We extracted its **learning system** — BM25+ keyword search over a knowledge base, feedback loops from task outcomes, idle-time "study sessions" where the agent reviews what went wrong. This is CashClaw's secret sauce for getting better over time.

Do we need it? YES — 8gent-code currently forgets everything between sessions. This gives it memory.

What we're MISSING: The actual autonomous freelance loop. CashClaw's killer feature is that it makes money while you sleep. We should consider adding a "bounty mode" where 8gent picks up GitHub issues with bounties (Gitcoin, Algora) and solves them.

Verdict: 🟢 MERGE the knowledge system. Consider adding bounty mode later as a separate feature.

---

2. `feature/nemoclaw-policy-engine`

Original Project: NemoClaw (NVIDIA) — A security wrapper for autonomous agents. Runs agents in sandboxed containers with Landlock/seccomp. All network requests go through a policy engine. Human approval for dangerous ops.

What NemoClaw actually does: Makes autonomous agents SAFE to run unattended. Without it, an always-on agent could delete your files, exfiltrate data, or run up API bills. NemoClaw is the seatbelt.

What we took: YAML-based policy system (allow/deny rules for filesystem, commands, network), approval gates for destructive ops, privacy-aware model routing (sensitive code stays local, general code can go to cloud).

Do we need it? YES, CRITICALLY — If 8gent-code is going to run autonomously (infinite mode, overnight competitions, always-on bot), it MUST have guardrails. This is table stakes for enterprise customers.

What we're MISSING: The actual container sandbox (Landlock/seccomp). We only have policy evaluation, not enforcement at the OS level. Fine for now, Docker integration later.

Verdict: 🟢 MERGE FIRST — This is the security foundation everything else sits on.

---

3. `feature/hermes-self-evolution`

Original Project: Hermes Agent (NousResearch) — "The agent that grows with you." A persistent, multi-platform agent that auto-creates reusable skills, maintains SQLite memory with full-text search, and uses GEPA (genetic evolution) to improve its own prompts.

What Hermes actually does: When it solves a hard problem, it writes a skill file so it can solve similar problems faster next time. It remembers you across sessions. It evolves its own system prompt using evolutionary algorithms. 9,100+ stars.

What we took: Auto-skill generation (solve hard task → write reusable YAML skill), SQLite+FTS5 persistent memory, tool self-registration (decorator pattern), checkpoint manager (snapshot git state before risky ops), reflection system (post-task "what worked / what didn't" journals).

Do we need it? YES — The skills system is how 8gent gets smarter at specific task types. The SQLite memory replaces the flat-file approach. Checkpoints prevent data loss during autonomous runs.

What we're MISSING: GEPA evolutionary prompt optimization. Hermes literally evolves its own prompts using genetic algorithms. We have the autoresearch mutation loop which is similar but less sophisticated.

Risks: Overlaps with the already-merged `memory-v2-core` branch. Need to deduplicate before merging.

Verdict: 🟡 MERGE WITH DEDUP — Great features but check for conflicts with existing memory system first.

---

4. `feature/openviktor-github-deep`

Original Project: OpenViktor — Reverse-engineered "AI employee" that lives in Slack, connects to 3,000+ tools, ingests all company context, and autonomously finds and executes work. Got taken down for IP issues.

What OpenViktor actually does: It's a full AI team member. It reads your Slack, email, docs, and code. It identifies tasks without being asked. It builds automations. It remembers everything about your organization. The 19 skill prompt files were the real value.

What we took: Deep GitHub integration (auto-create issues from benchmark failures, sync scores to pinned issues, project boards, auto-PRs with benchmark results), workflow automation (branch → commit → push → PR in one call), cron scheduling (GitHub Actions for nightly benchmarks), multi-provider LLM engine (Ollama → OpenRouter → Anthropic → OpenAI with cost tracking).

Do we need it? YES — James specifically asked for deeper GitHub integration. The multi-provider engine with cost tracking is essential for any serious tool.

What we're MISSING: The Slack integration and autonomous task discovery. OpenViktor's best feature was that it didn't wait for instructions — it found work on its own. We partially have this via the Telegram bot, but not at OpenViktor's level.

Verdict: 🟢 MERGE — Fills a real gap in GitHub workflow automation.

---

5. `feature/agent-browser-integration`

Original Project: agent-browser (Vercel Labs) — CLI tool for headless browser automation designed for AI agents. Snapshot-based element refs, not CSS selectors.

What agent-browser actually does: Gives AI agents eyes on the web. Open pages, click buttons, fill forms, take screenshots, extract text — all via CLI commands that agents can call as tools.

What we took: TypeScript wrapper around the `agent-browser` CLI. BrowserTools class for low-level ops (open, click, fill, screenshot). BrowserResearch class for high-level ops (web search, GitHub repo analysis, doc reading, local server testing).

Do we need it? YES — Without browser access, 8gent-code can't research docs, test web apps, or browse GitHub. Every serious coding agent needs web access.

What we're MISSING: The agent-browser CLI needs to be installed separately (`npm install -g agent-browser`). Should add auto-install to 8gent's onboarding.

Verdict: 🟢 MERGE — Essential capability, lightweight integration.

---

6. `feature/hypothesis-loop`

Original Project: codex-autoresearch — Autonomous code improvement loop. Makes a change, commits, verifies (tests/lint), keeps if pass, reverts if fail. Escalates: REFINE → PIVOT → web search. Persists lessons across runs.

What codex-autoresearch actually does: Every code change is a hypothesis. Commit it. Test it. Keep or revert. If it fails, try a different approach. If that fails, search the web for help. It never ships broken code because failures get auto-reverted.

What we took: The full hypothesis engine (apply → commit → verify → keep/revert), escalation strategy (REFINE 3x → PIVOT 2x → GIVE_UP), cross-run lessons file (`~/.8gent/lessons.jsonl`), self-healing executor from AutoResearchClaw (detect NaN/type errors → auto-fix → retry up to 10 rounds).

Do we need it? YES, THIS IS THE CORE DIFFERENTIATOR — This is what separates a toy agent from a professional one. Claude Code doesn't do this. Cursor doesn't do this. Aider doesn't do this. The hypothesis loop means 8gent never ships broken code.

What we're MISSING: Web search fallback (the PIVOT → WEB_SEARCH escalation). Browser integration (branch #5) would enable this.

Verdict: 🟢 MERGE — This is the single most important feature for competitive differentiation.

---

7. `feature/blast-radius-engine`

Original Project: code-review-graph — Local knowledge graph using Tree-sitter + SQLite. Maps function calls, inheritance, and test coverage. Reduces Claude's token usage 6.8-49x by computing the blast radius of changes.

What code-review-graph actually does: Instead of feeding the whole file to the LLM, it traces exactly which functions call the one you're changing, which tests cover it, and which interfaces constrain it. Feed only that minimal context.

What we took: SQLite-backed code graph (nodes = functions/classes/types, edges = calls/imports/tests), blast-radius queries ("if I change X, what breaks?"), impact estimation (low/medium/high/critical), prompt-injectable format.

Do we need it? YES — We already have `jcodemunch` for AST-based code exploration (97% token savings). This EXTENDS it with relationship tracking. 8gent-code uses local models with small context windows — every token saved matters.

Risks: Overlaps with jcodemunch. Uses regex parsing instead of Tree-sitter (less accurate but zero deps). Could produce stale graphs if not re-indexed after changes.

Verdict: 🟡 MERGE WITH CAUTION — Evaluate overlap with jcodemunch. Consider wiring blast-radius queries through jcodemunch's existing index instead of maintaining a separate SQLite DB.

---

8. `feature/worktree-swarm`

Original Project: ClawTeam — Multi-agent swarm framework. Agents spawn sub-agents in isolated git worktrees. Communication via filesystem-based inboxes. tmux windows for each agent.

What ClawTeam actually does: Decomposes a big task into subtasks, spawns a separate agent for each subtask in its own git branch (via worktrees), they work in parallel without conflicts, then results merge back. Like a team of developers working on feature branches simultaneously.

What we took: WorktreeAgentManager (spawn agents in isolated worktrees with auto-branch), filesystem inbox messaging (JSON files for inter-agent communication), TaskQueue (file-based priority queue), auto-PR creation from agent branches, GitHub Actions CI workflow for agent branches.

Do we need it? YES FOR SCALE — If 8gent-code wants to handle large projects, it needs parallel execution. One agent doing auth while another does tests while another does docs — all in isolated branches.

Risks: Most complex branch. Spawned processes could leak. Worktrees accumulate if not cleaned up. Race conditions on shared state. Security: spawned agents inherit all permissions.

What we're MISSING: tmux integration (visual monitoring of parallel agents). The NemoClaw policy engine (branch #2) should gate what spawned agents can do.

Verdict: 🟡 MERGE WITH CAUTION — Merge AFTER nemoclaw-policy-engine so spawned agents have guardrails.

---

9. `feature/remote-monitor-memory`

Original Project (remote): clsh — Streams real terminal sessions via WebSocket tunnels. Watch Claude Code work from your phone.

Original Project (memory): Nuggets — AI assistant with holographic memory. Facts recalled 3+ times auto-promote to permanent context.

What these actually do: clsh lets you open a browser on your phone and watch the agent typing in real-time. Nuggets ensures frequently-used facts become permanent knowledge — no vector DB, just frequency counting.

What we took: WebSocket terminal server (xterm.js viewer page, broadcast to all connected clients, optional localhost.run tunnel), frequency-based memory promotion (track recall count per fact, promote to `PERMANENT.md` after 3+ recalls across 2+ sessions).

Do we need it? YES — Remote monitoring is essential for overnight runs (you were asking for Telegram updates all night — this gives you a live view instead). Frequency memory is the simplest effective long-term memory system.

Verdict: 🟢 MERGE — Both features are lightweight, well-scoped, and solve real problems.

10-14. STALE BRANCHES (Already Merged)

These 5 branches have already been merged into main. The branch pointers are leftover:

Branch	Status	Action
`feature/knowledge-graph`	Already merged	`git branch -d`
`feature/memory-v2-core`	Already merged	`git branch -d`
`feature/session-convex-sync`	Already merged	`git branch -d`
`feature/stripe-billing`	Already merged	`git branch -d`
`feature/tenant-convex-persistence`	Already merged	`git branch -d`

Verdict: 🗑️ DELETE these branch pointers. The code is already in main.

TL;DR — The Honest Truth

Branch	Original Repo's Purpose	What We Actually Took	Need It?
cashclaw	**Make money autonomously**	Learning/memory system	Yes (but we skipped the money part)

Branch	Original Repo's Purpose	What We Actually Took	Need It?
nemoclaw	**Secure autonomous agents**	Policy engine + approval gates	YES — critical for safety
hermes	**Self-evolving persistent agent**	Skills + memory + reflections	Yes (dedup needed)
openviktor	**AI employee in Slack**	GitHub deep integration	Yes
agent-browser	**Browser for AI agents**	Web research + testing	Yes
hypothesis-loop	**Auto-revert broken code**	Commit-verify-revert loop	YES — #1 differentiator
blast-radius	**Token-efficient code review**	AST graph + impact queries	Yes (check jcodemunch overlap)
worktree-swarm	**Parallel agent teams**	Worktree isolation + messaging	Yes (merge last)
remote-monitor	**Watch agent from phone**	WebSocket terminal + memory	Yes

What we should STILL build (not yet on any branch):

1. **CashClaw's bounty mode** — auto-claim and solve GitHub bounties for money
2. **Hermes's GEPA** — evolutionary prompt optimization
3. **OpenViktor's autonomous task discovery** — agent finds work without being asked
4. **NemoClaw's container sandbox** — actual OS-level isolation, not just policy